

# MAXIMUS ERP

## Documentation technique de l'application

Architecture · Routage · Données · Permissions · Modules · Thème

# Documentation technique — MAXIMUS ERP

## 1. Vue d'ensemble

MAXIMUS ERP est une application web React/TypeScript construite dans un monorepo pnpm. Elle simule un ERP SaaS multi-entreprises avec :

- 1 un espace d'administration MAXIMUS ;
- 1 des espaces entreprise et employés ;
- 1 un catalogue de modules activables ;
- 1 une organisation hiérarchique par unités ;
- 1 des rôles et permissions par fonctionnalité ;
- 1 des écrans métier pour Commerce, Stocks, Présences et les modules opérationnels ;
- 1 une personnalisation visuelle par entreprise.

L'application MAXIMUS est principalement un prototype fonctionnel côté navigateur. Son état de démonstration est conservé dans `localStorage`.

## 2. Structure du monorepo

```
artifacts/  
├─ maximus/      # frontend React/Vite de MAXIMUS ERP  
└─ api-server/    # serveur API Express  
  
lib/              # packages partagés  
scripts/          # scripts de workspace et post-merge  
docs/             # documentation du projet
```

Le workspace est défini dans `pnpm-workspace.yaml`. Les packages situés dans `artifacts/*`, `lib/*` et `scripts` sont gérés par pnpm.

## 3. Frontend MAXIMUS

### 3.1 Entrée et rendu

- 1 `artifacts/maximus/src/main.tsx` monte l'application React.
- 1 `artifacts/maximus/src/App.tsx` orchestre la session, l'état global de démonstration, la navigation, le thème et les écrans.
- 1 `artifacts/maximus/src/index.css` contient les tokens visuels, les variables de thème et les règles responsive.
- 1 `artifacts/maximus/vite.config.ts` configure Vite, l'alias `@`, le chemin de base et le proxy `/api`.  
Vite attend les variables suivantes :
  - 1 `PORT` : port du serveur de développement ou de preview ;
  - 1 `BASE_PATH` : chemin de base de l'artefact.

Le serveur Vite écoute sur `0.0.0.0` et autorise les hôtes du preview Replit.

### 3.2 Routes applicatives

Le routage est assuré par Wouter :

- 1 `/` : connexion ;

<sup>1</sup> /inscription : inscription ou création administrative d'une entreprise ;

<sup>1</sup> /maximus/... : espace d'administration MAXIMUS ;

<sup>1</sup> /kora/... : espace entreprise et employés.

Les routes sont centralisées dans `artifacts/maximus/src/routes/app-routes.tsx` :

<sup>1</sup> AdminRouter sélectionne les écrans MAXIMUS ;

<sup>1</sup> KoraRouter sélectionne les écrans d'entreprise ;

<sup>1</sup> les routes sont filtrées par session, modules autorisés et permissions.

Les paramètres d'onglet sont transmis dans la query string, par exemple :

```
/kora/commerce?tab=products  
/kora/stocks?tab=inventory
```

Les routeurs comparent le chemin sans la query string. Les écrans utilisent `src/lib/query-tab.ts` pour lire l'onglet actif et gérer les alias historiques.

## 4. État et persistance

### 4.1 Store local

`artifacts/maximus/src/lib/store.ts` contient :

<sup>1</sup> les types métier ;

<sup>1</sup> le catalogue des modules ;

<sup>1</sup> les sous-rubriques Stocks ;

<sup>1</sup> les presets de secteurs d'activité ;

<sup>1</sup> les entreprises, employés, rôles et unités de démonstration ;

<sup>1</sup> les fonctions `loadData()` et `saveData()`.

La clé principale de persistance est :

```
maximus-data-v1
```

La lecture du store :

<sup>1</sup> tente de charger les données JSON existantes ;

<sup>2</sup> restaure la structure attendue ;

<sup>3</sup> complète les données héritées si nécessaire ;

<sup>4</sup> utilise les données de démonstration lorsque le store est absent ou invalide.

### 4.2 Session et préférences

Les informations suivantes sont conservées dans `localStorage` ou `sessionStorage` :

<sup>1</sup> `maximus-session` : session active ;

<sup>1</sup> `maximus-sidebar-collapsed` : état replié du menu ;

<sup>1</sup> `maximus-company-search` : recherche d'entreprise dans l'administration ;

<sup>1</sup> préférences de notification et de confirmation du module Stocks ;

<sup>1</sup> états locaux de Commerce par entreprise.

La synchronisation entre onglets repose sur l'événement navigateur storage.

## 5. Modèle d'accès et permissions

La chaîne de contrôle est :

Entreprise → unité → rôle → sous-fonctionnalité → action

### 5.1 Plafonds d'accès

- 1 `Company.allowedModules` définit les modules disponibles pour l'entreprise.
- 1 `OrgNode.moduleIds` réduit les modules accessibles à une unité.
- 1 `Role.modulePermissions` définit les permissions du rôle.
- 1 Les permissions détaillées utilisent des clés de fonctionnalité propres au module.

Le périmètre le plus restrictif l'emporte. Un rôle ne peut jamais réactiver un module retiré par l'entreprise ou par l'unité.

### 5.2 Calcul des permissions

`artifacts/maximus/src/lib/employee-permissions.ts` centralise notamment :

- 1 la reconstruction de l'ascendance de l'employé ;
- 1 la vérification du rattachement du rôle à l'unité ;
- 1 les permissions Commerce ;
- 1 les permissions détaillées Stocks ;
- 1 les permissions Présences ;
- 1 la résolution des dépendances entre fonctionnalités.

`artifacts/maximus/src/lib/permission-keys.ts` produit les clés canoniques et protège la résolution des dépendances contre les cycles.

Au niveau racine d'un module, l'éditeur propose seulement voir. Les actions créer et modifier sont configurées dans les sous-fonctionnalités.

## 6. Catalogue des modules

Les modules sont décrits comme des données structurées dans `store.ts`.

Un module contient notamment :

```
{
  id,
  name,
  description,
  features,
  featureDependencies,
  status
}
```

`getConfiguredModules()` applique les réglages administratifs :

- 1 `moduleOverrides` pour modifier le nom, la description ou les fonctionnalités ;
- 1 `removedModules` pour retirer un module ;
- 1 le statut d'activation ou de bêta.

## 6.1 Commerce

`src/lib/commerce-permissions.ts` définit les onglets canoniques de Commerce et conserve les alias historiques comme Clients, Devis, Commandes, Chiffre d'affaires et Facturation.

Les liens Commerce doivent toujours produire les onglets canoniques dans la query string.

## 6.2 Stocks

Les dix sous-rubriques Stocks sont conservées dans `stockSubmodules`. Elles sont utilisées par :

- 1 la navigation verticale des employés ;
- 1 l'éditeur de permissions ;
- 1 le calcul des permissions détaillées ;
- 1 le module Stocks.

Les opérations sensibles attendent une confirmation explicite avant leur exécution.

## 6.3 Presets de secteurs

`SectorPreset` permet à MAXIMUS de définir, pour un secteur d'activité :

- 1 les modules proposés ;
- 1 les fonctionnalités proposées pour chaque module via `moduleFeatures`.

La page MAXIMUS → Secteurs d'activité applique les prérequis en visibilité lorsqu'une fonctionnalité dépendante est sélectionnée. Cette configuration appartient au profil de secteur, pas à la structure interne d'une unité.

## 7. Organisation et employés

Les écrans d'organisation sont séparés par responsabilité :

- 1 `organization-structure.tsx` : hiérarchie des unités ;
- 1 `organization-roles.tsx` : rôles et permissions ;
- 1 `organization-employees.tsx` : employés et affectations ;
- 1 `company-organization.tsx` : conteneur du parcours Organisation ;
- 1 `organization-shared.tsx` : composants et modèles partagés.

Les unités utilisent `parentId` pour construire la hiérarchie. Les rôles et employés sont affectés à une unité via `sectorId`.

Les employés ayant plusieurs modules disposent d'une navigation verticale groupée par module. Les notifications restent accessibles depuis la cloche supérieure.

## 8. Thème entreprise

`src/lib/company-theme.ts` convertit les couleurs hexadécimales de l'entreprise en variables HSL :

- 1 `primaryColor` : couleur principale des boutons, liens et états actifs ;
- 1 `accentColor` : couleur d'accent ;
- 1 `sidebarColor` : fond du menu latéral ;
- 1 `sidebar-active` : état actif et survol du menu, dérivé de `primaryColor`.

Le thème est appliqué :

- 1 au document racine par `applyCompanyTheme()` ;
- 2 à l'app-shell courant ;
- 3 au style actif transmis au composant Sidebar.

Cette double application garantit que les composants CSS et les styles inline utilisent la même palette d'entreprise.

## 9. API et modules de données

Le package `artifacts/api-server` fournit un serveur Express séparé avec des scripts de build et de démarrage.

Le frontend Vite réserve le préfixe `/api` au proxy vers le serveur API. Les modules métier disposent aussi d'adaptateurs locaux :

- 1 `src/lib/stock-api.ts` pour les données et opérations Stocks ;
- 1 `src/lib/presence-api.ts` pour les workflows Présences ;
- 1 le module Commerce conserve son état local par entreprise lorsqu'il fonctionne en mode prototype.

Les données sensibles ou les secrets ne doivent jamais être ajoutés dans le code ni dans la documentation. Les variables d'environnement sont gérées par l'environnement Replit.

## 10. Tests et commandes

Installation :

```
pnpm install
```

Développement MAXIMUS :

```
pnpm --filter @workspace/maximus run dev
```

Typecheck :

```
pnpm run typecheck
pnpm --filter @workspace/maximus run typecheck
```

Tests de permissions :

```
pnpm --filter @workspace/maximus test
```

Build :

```
pnpm run build
pnpm --filter @workspace/maximus run build
```

Serveur API :

```
pnpm --filter @workspace/api-server run dev
```

## 11. Points d'attention pour les évolutions

Avant d'ajouter un module ou une fonctionnalité :

- 1 ajouter l'identifiant au catalogue typé ;
- 2 définir ses fonctionnalités et dépendances ;
- 3 créer ses clés de permission canoniques ;

- 4 l'ajouter à la navigation et aux routes ;
- 5 connecter l'entreprise, l'unité, le rôle et l'employé ;
- 6 protéger l'écran par les permissions effectives ;
- 7 ajouter les tests de parcours et de permissions ;
- 8 vérifier les query strings, les alias historiques et les états destructifs ;
- 9 vérifier le rendu dans les espaces entreprise et employés.

Les règles détaillées et décisions métier restent documentées dans `[replit.md](../replit.md)`.